

Kingdom of Saudi Arabia
Ministry of Education
Umm Al-Qura University
College of Computers
Department of Computer Science



Reel Cinema management system

Name	ID
Nora Hasan Assaedi	445007859
Asayel Al-Sharif	446008482
Tumador Ahmed ALmehmad	445008169
Samaa Al-Hussain Al Zaid Al Sharif	44611608
Hadeer Tawfiq Almoulde	446001914

Group: 2

Instructor: Dr Uhood Al-Zahrani

Proposal:

This project aims to develop a complete cinema management system that simplifies the process of browsing movies, selecting rooms, reserving seats, and purchasing tickets. The system allows the user to view movies by genre, see the room assigned to each movie, and choose a seat through a clear seat-map display. It also supports buying both normal and VIP tickets with automatic price calculation, in addition to selecting snacks such as drinks and popcorn. The system is designed using Object-Oriented Programming principles to ensure clarity, organization, and easy future development.

Menu hierarchy:

The main menu of the system consists of the following:

1. Login
2. Add new movie
3. View all movies
4. Search movies
5. Update movie info
6. Sell ticket (with snacks/drinks)
7. Just a snack
8. Reserve seat only
9. Empty seats in movie of your choosing
10. Exit

Design explanation:

Our Cinema management project has 10 classes that start with a welcome and then a login with a fixed username and password, with three tries each before the program stops. After typing in the correct username and password a menu will pop up about managing the cinema like updating movies, searching for movies, reserving seats, selling tickets etc.

In our project we've decided for our Genre class to be our enumeration class because we wanted the genres to be fixed, and the popularity, age rating and pricing of each genre to be fixed.

Regarding our project, "Reel Cinema" there are relationships between classes, for example:

For aggregation:

Cinema classes aggregate Room and Movie class.

Movie class aggregate Room class and Genre class.

Ticket class aggregate Movie class and Drink class and Popcorn class.

Snacks aggregates Drink class and Popcorn class.

For inheritance:

Popcorn and Drinks class inheritance from Snacks class, because popcorn is a snacks and a drink is a snack.

And VIP Ticket class inheritance Ticket class so we can override the price and make a discount.

Description on how we tested it:

Ticket Class Testing

To verify the Ticket class functionality, a ticket was created using a Movie with a fixed price of 25, along with a Drink priced at 10 and a Popcorn priced at 15.

The ticket details were displayed to confirm that the information and total price were stored correctly.

Input values: Movie price: 25, Drink price: 10 ,Popcorn price: 15

Expected behaviours: The calculated total should equal the sum of the movie, drink, and popcorn prices.

Actual outcome: The total price was calculated as 50, and all ticket details appeared correctly.

VIPTicket Class Testing

The VIPTicket class behavior was checked by converting an existing Ticket into a VIP ticket and displaying its details.

Input values: Original ticket total: 50

Expected behaviours: A discount should be applied, resulting in a reduced total price of 37.5.

Actual outcome: The discount was applied successfully, and the final price displayed was 37.5.

Cinema Class Testing

Input values: The application was launched without providing any user input.

Actual outcome: A Cinema object was initialized automatically, and the option to view all movies was selected.

Expected behaviours: All predefined movies were shown correctly and organized according to their genres.

Movie Class Testing

Input: Movie title: Avatar, Genre: Science Fiction, Room number: 7

Actual outcome: A Movie object was created using the provided data, and its information was displayed.

Expected behaviours: The movie name, genre, and age rating appeared correctly without issues.

CinemaDemo Class Testing

Input values: Menu selection: Add Movie, Movie title: Test Movie, Genre: Comedy, Room number: 5

Actual outcome: The movie was added using the menu options, followed by displaying the movie list.

Expected behaviours: The newly added movie appeared successfully in the list.

Seat Reservation Test Case

Input values: Row: 2, Column: 3

Actual outcome: The seat reservation feature was used to reserve the selected seat.

Expected behaviours: The seat status was updated from available (O) to reserved (X).

File Writing Test Case

Input values: Feedback message: The system works well

Actual outcome: The program was exited, triggering the feedback-saving process.

Expected behaviours: A file named Feedback.txt was created, and the feedback text was saved correctly.

Drinks Class Testing

To test the Drinks class, a drink object was created with the name “Pepsi” and size set to “Large”.

The size was later modified to “Medium” to observe any change in pricing.

Input values: Name: Pepsi, Size: Large → Medium

Expected behaviours: The drink price should update automatically based on the selected size.

Actual outcome: The price was 12.0 SAR for the large size and updated to 8.0 SAR after changing to medium.

Popcorn Class Testing

The Popcorn class was examined by creating a popcorn object with flavor “Caramel” and size “Small”, then updating the size to “Large”.

Input values: Flavour: Caramel, Size: Small → Large

Expected behaviours: The popcorn price should adjust according to the chosen size.

Actual outcome: The price was 7.0 SAR for the small size and increased to 15.0 SAR when set to large.

Genre Enum Testing

The Genre enumeration was checked by selecting the “ROMANTIC” genre and retrieving its related data.

Input values: Genre: ROMANTIC

Expected behaviours: The enum should return the correct ticket price and age restriction.

Actual outcome: The returned values were a price of 20.0 SAR and an age rating of 13.

Room Class Testing

The Room class was verified by initializing a room with number 3.

Input values: Room number: 3

Expected behaviours: The assigned room number should be stored and accessible.

Actual outcome: The room number was saved correctly and returned as 3.

Seats Class Testing

The Seats class was evaluated using the default 5×5 seating layout.

Test Case 1: Reserving an empty seat

Input: Row = 1, Column = 2

Expected behaviours: The selected seat should be marked as reserved (X).

Actual outcome: The seat was reserved successfully and shown as (X).

Test Case 2: Reserving the same seat again

Input values: Row = 1, Column = 2

Expected behaviours: The system should block the action and display a warning.

Actual outcome: A warning message appeared indicating the seat was already taken.

Test Case 3: Entering invalid seat numbers

Input: Row = 10, Column = 3

Expected behaviours: An error message should appear, and no reservation should occur.

Actual outcome: An error message was shown, and the seat status remained unchanged.

Snacks Class Testing

The Snacks class was tested by creating a snack that combines a drink and popcorn.

Test Case 1: Snack creation

Input values: Drink: Coca Cola (Medium), Popcorn: Cheese (Large)

Expected behaviours: Snack details and total price should be displayed correctly.

Actual outcome: All snack information appeared correctly, and the total price was calculated accurately.

Test Case 2: Total price calculation

Input values: Drink price + Popcorn price

Expected behaviours: The total should equal the sum of both prices.

Actual outcome: The calculated total matched the expected value.

UML diagram:

Conclusion and Simplified Idea of the Project:

The system developed in this project successfully achieved its main goal and met all the required specifications. We implemented a complete program based on Object-Oriented Programming (OOP) principles, which were effectively applied throughout the project, including class structure design and data processing operations.

All the required OOP concepts were applied, such as Inheritance, Encapsulation, Polymorphism, Exception Handling, the use of ArrayList, Object Creation, this keyword, Method Overloading, and Text File Handling. These concepts helped make the system organized, flexible, and easy to maintain and develop in the future. From this project, we learned how to design and develop a complete program from scratch using Java.

We also identified several future improvements for the system, including:

1. Adding a graphical user interface to enhance user experience.
2. Connecting the system to a database for secure and organized data storage.
3. Implementing an advanced login system with different user roles and permissions.
4. Creating reports and statistics to analyse system data.
5. Improving search algorithms.

Supporting multiple languages.

Group participation:

Asayel	In report: Project Documentation & Structure (Project title, team members, proposal, menu description, report structure, task distribution page). In program: Created the Seats class and Main class Worked on the UML diagram
Tamader	In report:UML Diagram & Menu Hierarchy (Drawing the full UML class diagram, showing relationships and multiplicity, creating the menu hierarchy tree) organized the project report In program: Created the Main class Worked on Snacks, Popcorn, and Drinks classes
Nora	In report: Design Explanation (Explaining class design, enum usage, relationships between classes). In program: Created the Cinema, Movie, Genre, Ticket, VIPTicket, and Room classes Created the Main class
Samaa	In report: Testing (Testing all classes: Seats, Movie, Ticket, VIPTicket, Snacks, Cinema + room+ File Input testing). In program: Their contribution focused on explaining the code, and they were not involved in the writing or development process Movie class
Hadeer	In report: Main Menu & Integration Code (Building the main menu, connecting all classes, implementing core operations, running the full system) In program: Their contribution focused on explaining the code, and they were not involved in the writing or development process Room class

